

1. Describe how you could create an efficient algorithm based on quicksort for the following problem. A simple way to accomplish this task would be to sort the array completely, but your algorithm can be more efficient if you do not attempt to completely sort the list! Given a list of n distinct positive integers, partition the list into two subsets, each of size $n/2$, such that the difference between the sums of the integers in the two subsets is maximized. You may assume that n is a multiple of 2 (i.e. n is even). Although you need not give a precise algorithm (unless you like to do so), determine its time complexity based on the master theorem.

one of
→ smallest & 1/2
and other
1/2 larger
numbers

• Quicksort currently:

- Has a pivot to split into 2 arrays recursively
- Does swaps
- Does not need to recombine as sorted in place.

• New Algorithm:

- Needs the higher HALF of the values in the higher indices. It does not matter which element it is, as long as it is n one above $\frac{n}{2}$.
- Vice versa for the lower half of the values.

EX $1, 2, 3, 4, 5, 6, 7, 8, 9, 10$
 $\downarrow$
 $5, 3, 2, 1, 4, 9, 10, 8, 6, 7$
 $\downarrow$
 $9, 10, 8, 7, 6, 5, 3, 2, 1, 4$

All are acceptable for maximum difference. If it needs to be positive, only top 2 are acceptable

Would use Hoare's Partition to find the $\frac{1}{2}^{\text{th}}$ largest element, and doing swaps after crossing.

(In the book this is given through the Hoare algorithm)

Partition for myself

```

1  x = A[p]
2  i = p - 1
3  j = r + 1
4  while TRUE
5      repeat
6          j = j - 1
7          until A[j] ≤ x
8      repeat
9          i = i + 1
10         until A[i] ≥ x
11     if i < j
12         exchange A[i] with A[j]
13     else return j
    
```

trying to find something larger or smaller than pivot

EX $n = 22$ $\frac{1}{2} = 11$
 $A = [1, 2, \dots, 22]$
 $x = A[11]$
 $i = 1$
 $j = 22$
 $\dots$
 $j = 22, 21, \dots$

1) Set a pivot value to meet? check to see if each element is larger or smaller than the pivot

2) If values are found, then exchange them

3) Repeat until Array is mostly sorted when there is a cross of i and j
 (highest half is on the right and lowest half is left)

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$a = 1$$

$$b = 2$$

(2 halves)

$$D(n)$$

(worst case needs $o(1)$?)

(1 for each partition)

(1 for each part → 4/1)

$$n^{\log_b a} = n^{\log_2 1} \rightarrow n^0 = 1 \leq O(n)$$

Driving Function Dominates, $T(n) = O(n)$

2. Use Strassen's algorithm to compute the matrix product (no credit if you do not show ALL steps or if it becomes clear you have not used Strassen's algorithm):

$$\begin{matrix} A \\ B \end{matrix} \begin{bmatrix} 2 & 1 \\ 4 & 2 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 7 & 5 \end{bmatrix}$$

I hope you don't mind me entering values in what is given in the book.

Now, let's delve into the details.

$$\begin{aligned} S_1 &= B_{12} - B_{22}, & 4 - 5 &= -1 \\ S_2 &= A_{11} + A_{12}, & 2 + 1 &= 3 \\ S_3 &= A_{21} + A_{22}, & 4 + 2 &= 6 \\ S_4 &= B_{21} - B_{11}, & 7 - 3 &= 4 \\ S_5 &= A_{11} + A_{22}, & 2 + 2 &= 4 \\ S_6 &= B_{11} + B_{22}, & 3 + 5 &= 8 \\ S_7 &= A_{12} - A_{22}, & 1 - 2 &= -1 \\ S_8 &= B_{21} + B_{22}, & 7 + 5 &= 12 \\ S_9 &= A_{11} - A_{21}, & 2 - 4 &= -2 \\ S_{10} &= B_{11} + B_{12}, & 3 + 4 &= 7 \end{aligned}$$

$$\begin{aligned} P_1 &= A_{11} \cdot S_1 (= A_{11} \cdot B_{12} - A_{11} \cdot B_{22}), & 2 \cdot -1 &= -2 \\ P_2 &= S_2 \cdot B_{22} (= A_{11} \cdot B_{22} + A_{12} \cdot B_{22}), & 3 \cdot 5 &= 15 \\ P_3 &= S_3 \cdot B_{11} (= A_{21} \cdot B_{11} + A_{22} \cdot B_{11}), & 6 \cdot 3 &= 18 \\ P_4 &= A_{22} \cdot S_4 (= A_{22} \cdot B_{21} - A_{22} \cdot B_{11}), & 2 \cdot 4 &= 8 \\ P_5 &= S_5 \cdot S_6 (= A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22}), & 4 \cdot 8 &= 32 \\ P_6 &= S_7 \cdot S_8 (= A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22}), & -1 \cdot 12 &= -12 \\ P_7 &= S_9 \cdot S_{10} (= A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12}), & -2 \cdot 7 &= -14 \end{aligned}$$

$$C_{11} = \cancel{C_{11}} + P_5 + P_4 - P_2 + P_6, \quad 32 + 8 - 15 - 12 = 13$$

$$C_{12} = \cancel{C_{12}} + P_1 + P_2, \quad -2 + 15 = 13$$

$$C_{21} = \cancel{C_{21}} + P_3 + P_4, \quad 18 + 8 = 26$$

$$C_{22} = \cancel{C_{22}} + P_5 + P_1 - P_3 - P_7, \quad 32 - 2 - 18 + 14 = 26$$

$$C = \begin{bmatrix} 13 & 13 \\ 26 & 26 \end{bmatrix}$$

3. As described, Strassen's algorithm works for exact powers of 2. We could modify Strassen to multiply two $n \times n$ matrices, where n is not an exact power of 2 as follows: We add as many rows and columns as needed to turn the matrix into a larger square matrix, whose number of rows and columns would be the next higher power of 2, and extract the first n rows and columns that the resulting algorithm also runs in $\Theta(n^{\lg 7})$ time.

$$\begin{bmatrix} a & b & Q \\ c & d & Q \\ Q & Q & Q \end{bmatrix} \times \begin{bmatrix} a & b & e \\ c & d & f \\ g & h & i \end{bmatrix}$$

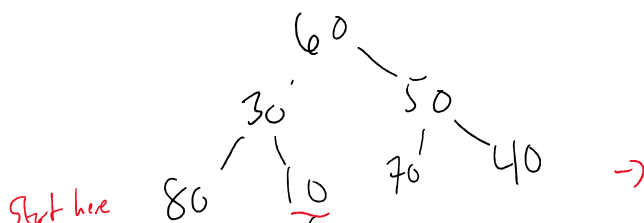
Adding Q s will be proportional to the perimeter increasing size by $(\# \text{ of rows} + \# \text{ of columns} - 1)$ elements.

of elements added are then added within a constant factor.

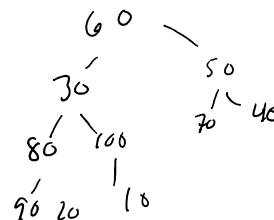
and that keeps the equation at $\Theta(n^{\lg 7})$

4. Show how the Heapsort algorithm works on the following numbers, by showing BOTH the binary tree view AND the array view at the beginning AND after each step of Build-Max-Heap, AND for the rest of Heapsort.
[Hint: (i) Do not get sloppy as you approach the end; show your work completely. (ii) Recall that the "parent", "left", and "right" functions on Page 162 link the two views; so, if your tree is not a "complete" binary tree, your array view would be weird with blank elements!]

[60 30 50 80 10 70 40 90 20 100]



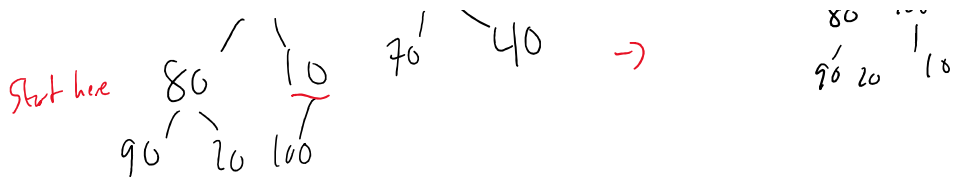
[60, 30, 50, 80, 100, 70, 40, 90, 20, 10]



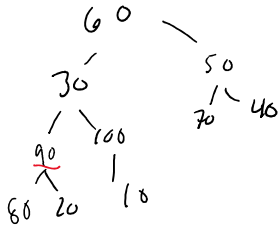
Strassen's algorithm to
 many zeroes at the end of
 number of rows (and
 from the result! Argue

We can see that Strassen's remarkable algorithm, comprising steps 1–4, produces the correct result using 7 submatrix multiplications and 18 submatrix additions. We can also see that recurrence (4.10) characterizes the complexity of the algorithm. Since [Section 4.5](#) shows that this recurrence has the solution $T(n) = \Theta(n^{\lg 7}) = o(n^3)$, Strassen's method is asymptotically faster than the $\Theta(n^3)$ MATRIX-MULTIPLY and MATRIX-MULTIPLY-RECURSIVE procedures.

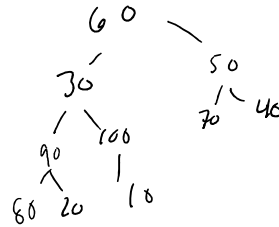
matrix product using 7
terizes its running time.
od asymptotically beats



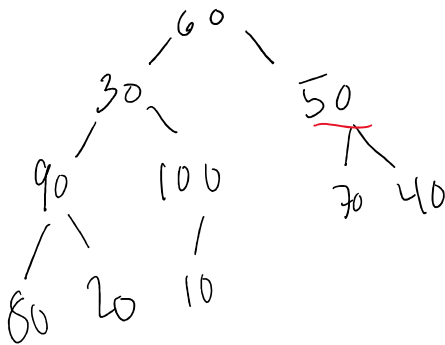
[60, 30, 50, 90, 100, 70, 40, 80, 20, 10]



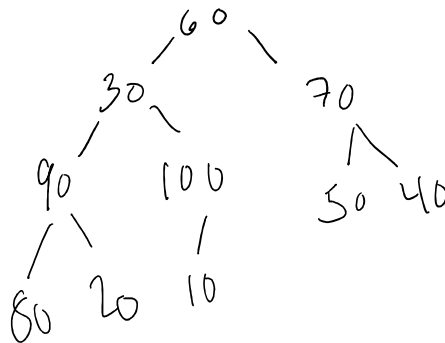
[60, 30, 50, 90, 100, 70, 40, 80, 20, 10]



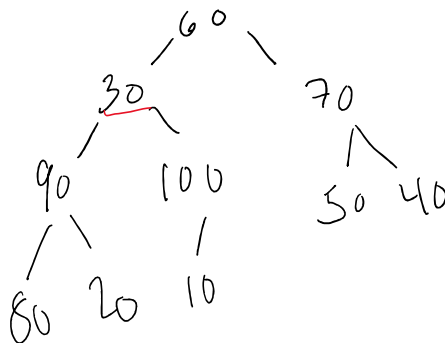
[60, 30, 50, 90, 100, 70, 40, 80, 20, 10]



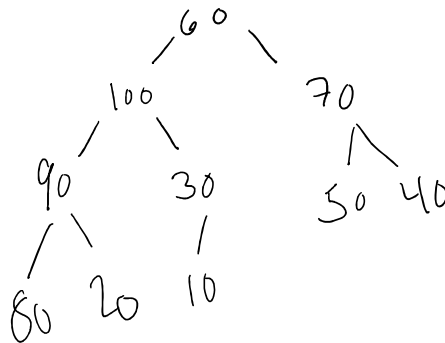
[60, 30, 70, 90, 100, 50, 40, 80, 20, 10]



[60, 30, 70, 90, 100, 50, 40, 80, 20, 10]



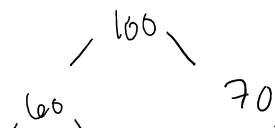
[60, 100, 70, 90, 30, 50, 40, 80, 20, 10]

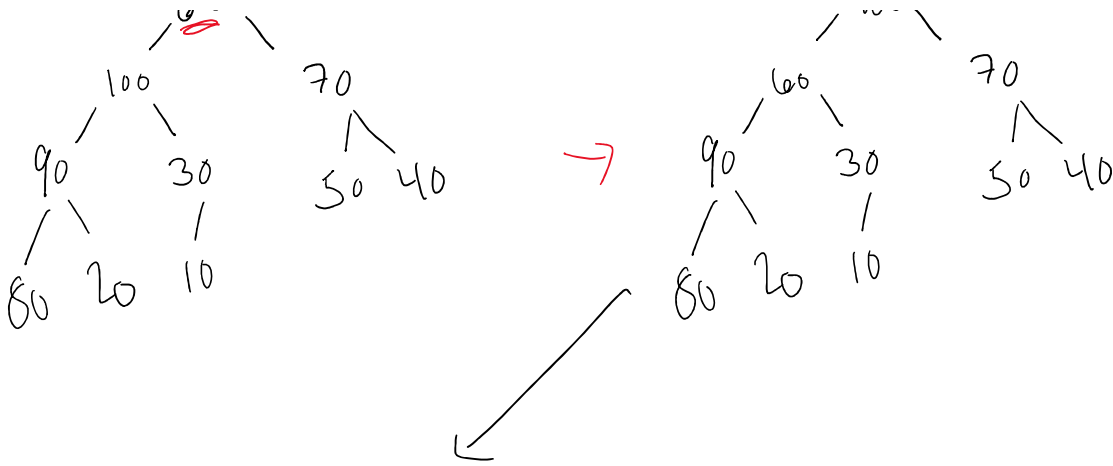


[60, 100, 70, 90, 100, 50, 40, 80, 20, 10]

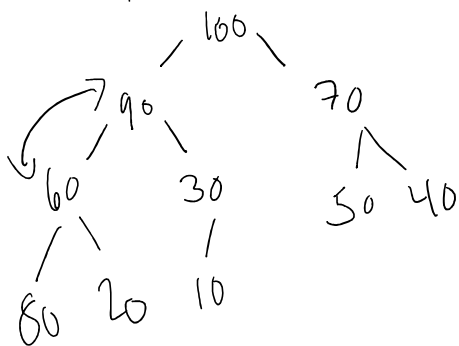


[60, 100, 70, 90, 100, 50, 40, 80, 20, 10]

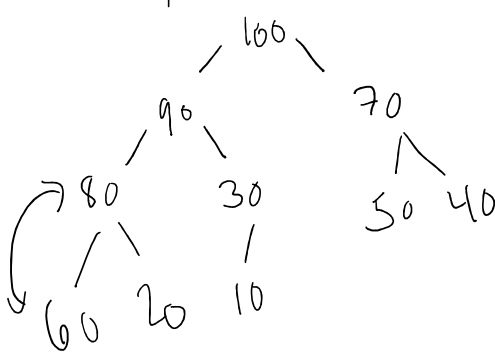




[100, 90, 70, 60, 100, 50, 40, 80, 20, 10]



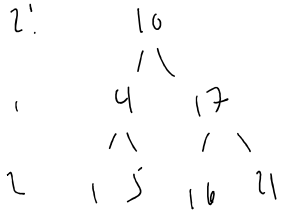
[100, 90, 70, 80, 100, 50, 40, 60, 20, 10]



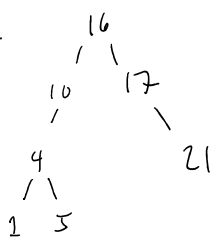
5. Exercise 12.1-1, page 289 of your text.

For the set of keys {1, 4, 5, 10, 16, 17, 21}, draw binary search trees of height 2, 3, 4, 5, and 6.

Height 2:

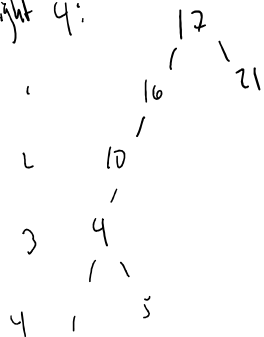


Height 3:

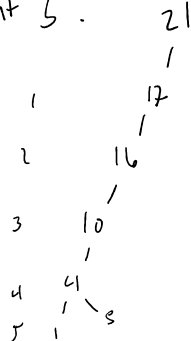


Height 4:

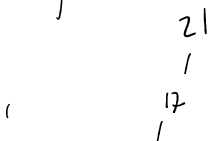
Height 4:



Height 5:



Height 6:



2 16
3 10
4 5
5 4
6 1

6. a) (CS 4120 ONLY) Is 5^n little- o , Θ , or little- ω of 7^n ? Justify your answer mathematically. NOW, take the natural (not binary) logarithms on both sides, and answer whether $\ln 5^n$ is little- o , Θ , or little- ω of $\ln 7^n$. Once again, justify your answer mathematically. (You may still check out b for learning better.)

a) Need to know the limits of the factors:

For Θ the limit would need to be $\mathbb{Q}$.

$$\text{So: } \lim_{n \rightarrow \infty} \frac{(5^n)}{(7^n)} = \lim_{n \rightarrow \infty} \left(\frac{5}{7}\right)^n = 0$$

So 5^n would be little- o of 7^n

which means that 5^n can not be little- ω or Θ .

b) If you take the natural log of each, they would become:

$$n \ln(5) \text{ and } n \ln(7)$$

The answer (change) as they'd be within a constant factor.

$$\lim_{n \rightarrow \infty} \frac{n \ln(5)}{n \ln(7)} \rightarrow \lim_{n \rightarrow \infty} \frac{n \ln(5)}{n \ln(7)} = \lim_{n \rightarrow \infty} \frac{\ln(5)}{\ln(7)} = \frac{\ln(5)}{\ln(7)}$$

So $n \ln(5)$ would be Θ of $n \ln(7)$

